

Vellore Institute of Technology, Vellore

[School / Department name]

Digital Watermarking and Steganography

Course Code: [Course Code] Slot: [Slot] Semester: [Semester]

FINAL PROJECT REPORT

AuthMark

AI Image Authenticity Verification Using Invisible Digital Watermarking

---

Submitted by

Name	Registration No.
Gaurish Todi	23BCI0262
Prakamya Bhuvi	23BCI0256

Under the guidance of

Dr. Diviya M.

Faculty, Vellore Institute of Technology, Vellore

GitHub Repository: https://github.com/GAURISHTODI/-AuthMark-AI-Image-Authenticity-Verification.git

October 2026

Abstract

The rapid spread of generative AI has made it easy to produce photo-realistic images that are indistinguishable from camera photographs. Metadata-based provenance tags are lost when an image is screenshotted or re-shared, and small local edits leave no visible trace. This report presents AuthMark, an end-to-end system that signs an AI-generated image with an invisible digital watermark and later verifies whether the image is authentic, edited, or unsigned.

AuthMark embeds a 96-bit payload (creator ID, timestamp and an HMAC-SHA256 signature) in the luminance channel using a one-level Haar discrete wavelet transform, 8x8 block discrete cosine transform and quantization index modulation (QIM). Because the payload is repeated over the whole image and shuffled by a secret key, it is recovered by soft majority voting, and blocks that disagree with the recovered payload reveal where an image was edited. On eight 512 x 512 test images the mark is invisible (mean PSNR 42.30 dB, SSIM 0.9828), survives JPEG compression down to quality 50, Gaussian noise up to sigma 8, rescaling and light blur, detects all localized edits tested, and produced no false accepts. The system is delivered as a command line tool, a Streamlit web application and a printable verification report.

Keywords: digital watermarking, image authenticity, DWT, DCT, quantization index modulation, HMAC, tamper localization, AI-generated images.

List of Figures

Figure 1: End-to-end architecture of AuthMark
Figure 2: Embedding pipeline (DWT, DCT, QIM)
Figure 3: Verification and tamper-localization flow
Figure 4: Original, signed, difference map and tampered images
Figure 5: Robustness of the watermark under common attacks
Figure 6: Web application, Sign tab (upload and signed output)
Figure 7: Web application, Sign tab results with PSNR and SSIM
Figure 8: Web application, Verify tab showing an AUTHENTIC verdict
Figure 9: Verify tab showing a TAMPERED verdict with the tamper map
Figure 10: Verification report tables in the web application
Figure 11: Saving the report and tamper map to a folder
Figure 12: Generated report for the tampered image and the text report

List of Tables

Table 1: Scope and boundary
Table 2: Structure of the 96-bit payload
Table 3: Verdict decision rule
Table 4: Technology stack
Table 5: Source modules
Table 6: Imperceptibility results
Table 7: Robustness results
Table 8: Tamper-detection results

Table of Contents

Abstract 1
1. Introduction 3
1.1 Background 3
1.2 Problem Statement 3
1.3 Motivation 3
2. Aim, Objectives and Scope 3
2.1 Aim 3
2.2 Objectives 3
2.3 Scope and Boundary 4
3. Background and Literature Review 4
3.1 Digital Watermarking 4
3.2 Related Work 4
3.3 Contribution of This Work 4
4. System Design 5
4.1 Architecture 5
4.2 Payload Structure 5
4.3 Security Model 5
5. Methodology 6
5.1 Embedding 6
5.2 Quantization Index Modulation 6
5.3 Extraction and Verification 7
5.4 Tamper Localization 7
5.5 Verdict Rule 8
6. Implementation 8
6.1 Technology Stack 8
6.2 Source Modules 8
7. Results and Discussion 8
7.1 Test Setup 8
7.2 Imperceptibility 9
7.3 Robustness 10
7.4 Tamper Detection 10
7.5 Web Application Demonstration 11
7.6 Discussion and Limitations 13
8. Conclusion and Future Work 13
References 13
Appendix A: Reproduction Steps 14

1. Introduction

1.1 Background

Digital watermarking hides information inside a digital signal so that the information travels with the content. Unlike metadata or a visible logo, an invisible watermark lives in the pixel values themselves and can survive format conversion, compression and moderate editing. Watermarking is traditionally used for copyright protection, ownership proof and content authentication [1], [2].

Generative models such as diffusion networks now produce images that people and basic detectors cannot reliably separate from real photographs. This makes provenance, meaning knowing who made an image and whether it has been changed, an important security problem.

1.2 Problem Statement

- **Realism:** AI-generated images look real, so appearance alone cannot establish origin.
- **Fragile provenance:** metadata and content credentials such as C2PA [8] are removed by screenshots and re-uploads.
- **Silent edits:** a swapped face or removed object leaves no visible clue and no proof of where the change was made.
- **No lightweight proof:** there is no simple way to show that an image came from a given creator and is unchanged since creation.

There is therefore a need for a mark that is invisible, cannot be forged without a secret, survives normal sharing, and shows where an image was edited.

1.3 Motivation

Social media, journalism and legal evidence all depend on trustworthy images. A watermark that carries a cryptographic signature gives creators a way to prove authorship, and gives reviewers a way to detect manipulation without needing the original file.

2. Aim, Objectives and Scope

2.1 Aim

To build an end-to-end system that signs AI-generated images with an invisible, key-protected watermark, and later verifies the image's authenticity and pinpoints tampered regions.

2.2 Objectives

- **Invisibility:** keep the signed image visually identical (PSNR above 40 dB).
- **Unforgeability:** protect the payload with an HMAC-SHA256 signature tied to a secret key.
- **Robustness:** survive JPEG compression, noise, brightness change, rescaling and light blur.
- **Tamper localization:** show which regions of an image were edited.
- **Usability:** provide a command line tool, a web interface and a printable report.

2.3 Scope and Boundary

In scope	Out of scope
Still images (PNG, JPEG)	Cropping, rotation and other geometric attacks
96-bit signed payload, single secret key	Strong blur (radius 1.0 and above)
Benign processing and localized edits	Full regeneration of the image by a diffusion model
Blind verification (original image not needed)	Video and audio; public-key verification

Table 1: Scope and boundary of the project

3. Background and Literature Review

3.1 Digital Watermarking

Watermarks are classified as visible or invisible, and as robust (survive processing), fragile (break on any change) or semi-fragile. Embedding can be done in the spatial domain or in a transform domain. Transform-domain methods such as the discrete cosine transform (DCT) [11] and the discrete wavelet transform (DWT) [10] spread the mark over frequency components, which makes it more robust and less visible than changing raw pixels [1], [3].

3.2 Related Work

- **Spread-spectrum watermarking [1]:** embeds a pseudo-random sequence in significant frequency components; a classic robust approach.
- **DCT-domain watermarking [3]:** embeds in selected block DCT coefficients, also the basis of JPEG-compatible methods.
- **Quantization index modulation [2]:** embeds bits by quantizing a coefficient onto one of two interleaved lattices, a blind and provably good scheme.
- **Deep-learning watermarking, HiDDeN [4]:** trains an encoder and decoder network end to end; robust but needs training and is hard to explain.
- **Generator-integrated marks [5], [6]:** Stable Signature fine-tunes a latent diffusion decoder, and Tree-Ring watermarks alter the initial noise; both require control of the generator.
- **Content credentials [8]:** C2PA attaches signed metadata to a file, but the metadata is lost when pixels are re-encoded or screenshotted.
- **Message authentication [7]:** HMAC provides keyed integrity and authenticity for short messages.

3.3 Contribution of This Work

The embedding technique (DWT, DCT and QIM) is established. The contribution of AuthMark is the way the pieces are combined for AI image authenticity:

1. **Signed payload inside the watermark:** creator ID and timestamp are protected by an HMAC tag, so a valid mark proves origin.
2. **Tamper localization from the same mark:** the payload is tiled over the image and blocks vote, so disagreeing blocks give a tamper map without a second fragile watermark or the original image.

3. **Three-way verdict:** AUTHENTIC, TAMPERED or NOT VERIFIED, separating benign processing from malicious edits.
4. **Gain search:** a small search over brightness gain restores QIM decoding after contrast changes.
5. **Key-seeded block permutation:** bit positions are unknown without the key.
6. **Complete system:** command line, web application, report generator, benchmark and unit tests.

4. System Design

4.1 Architecture

AuthMark has two services that share one secret key. The signing service embeds the payload into an image and outputs a lossless PNG. The image then travels through an uncontrolled channel (sharing, compression, resizing, editing). The verification service extracts the payload, checks the signature and compares every block against the recovered payload, then returns a verdict.

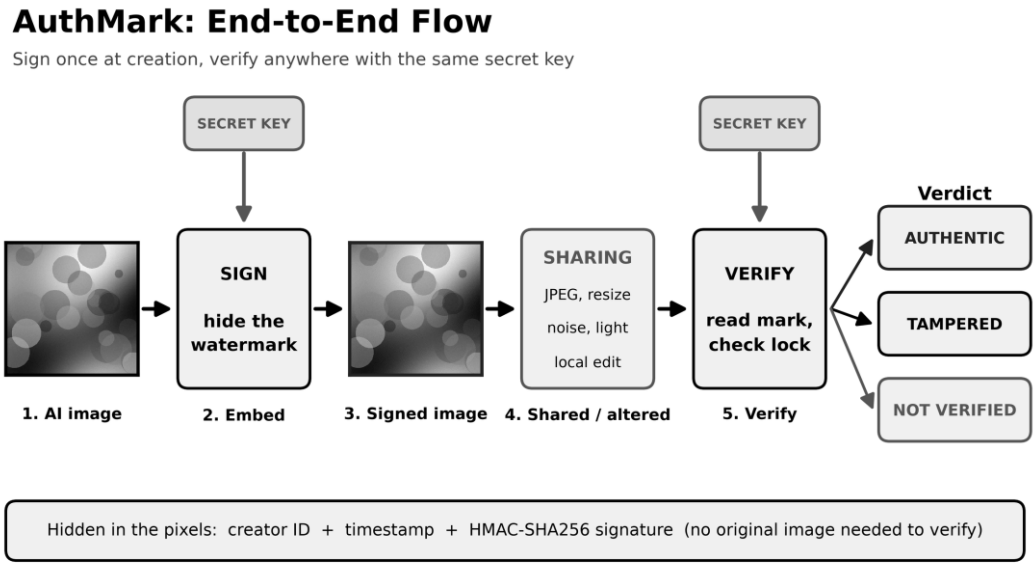


Figure 1: End-to-end architecture of AuthMark

4.2 Payload Structure

The payload is a 96-bit record. The signature is the first 32 bits of HMAC-SHA256 computed over the creator ID and timestamp with the secret key.

Field	Size	Description
Creator ID	32 bits	Identifier of the creator or model
Timestamp	32 bits	Unix time of signing
Signature	32 bits	Truncated HMAC-SHA256 of ID and timestamp

Table 2: Structure of the 96-bit payload

4.3 Security Model

The secret key is used twice. First, it seeds the pseudo-random permutation that assigns each payload bit to image blocks, so the bit positions cannot be located without the key. Second, it keys

the HMAC, so a valid payload cannot be forged. An image without a mark, or checked with a wrong key, yields a payload whose signature does not match and is reported as NOT VERIFIED. A 32-bit tag gives a random-forgery probability of about 2^-32 per attempt, which is adequate for a demonstration; a longer tag or a public-key signature is listed as future work.

5. Methodology

5.1 Embedding

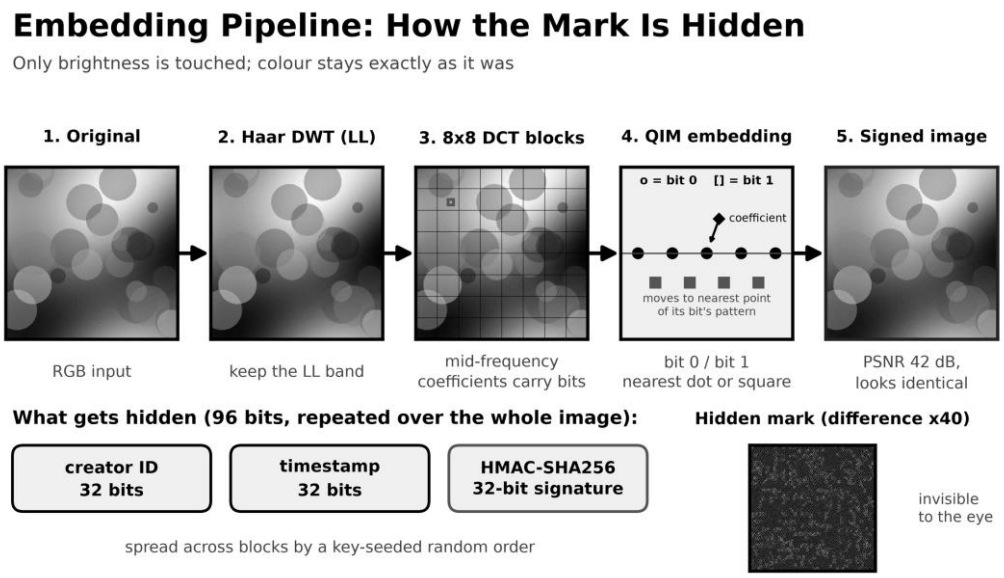


Figure 2: Embedding pipeline: DWT, DCT and QIM

Embedding proceeds in the following steps:

1. Convert the RGB image to YCbCr and take the luminance channel Y; Cb and Cr are left untouched so colour is preserved.
2. Pad Y to a multiple of 16 pixels and apply a one-level orthonormal Haar DWT; the LL (approximation) band is used for embedding.
3. Split the LL band into 8x8 blocks and apply a 2-D DCT to each block.
4. Build the 96-bit payload and assign each bit to blocks through a key-seeded permutation (about 10 blocks per bit for a 512 x 512 image).
5. In every block, embed the assigned bit into six mid-frequency coefficients by QIM (Section 5.2).
6. Apply the inverse DCT and inverse DWT, merge Y' with the original Cb and Cr, and save as a lossless PNG.

5.2 Quantization Index Modulation

A coefficient c carrying bit b in {0, 1} is replaced by c' = round((c - b*D/2) / D) * D + b*D/2, where D is the quantization step (D = 28 in this work). Bit 0 snaps to multiples of D and bit 1 to multiples of D offset by D/2. At extraction, the distances to the two lattices are compared and the closer lattice gives a soft vote in the range -1 to +1. A larger D gives more robustness and less invisibility. The coefficients used per block are (3,4), (4,3), (2,5), (5,2), (3,3) and (2,4), chosen from the mid-frequency band as a compromise between survival under compression and visibility.

5.3 Extraction and Verification

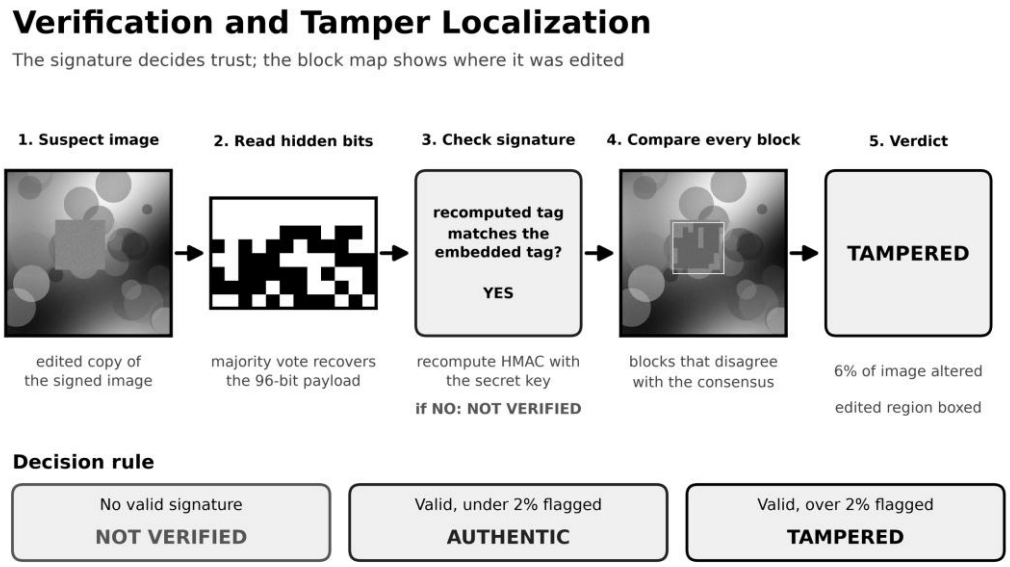


Figure 3: Verification and tamper-localization flow

1. Repeat the transform chain (YCbCr, Y, Haar DWT, block DCT) on the received image.
2. Search a global gain from 0.75 to 1.30 and keep the one that best fits the QIM lattice; this compensates for brightness or contrast scaling.
3. Sum the soft votes of all blocks assigned to each bit and take the sign to obtain the 96 payload bits.
4. Recompute the HMAC from the decoded creator ID and timestamp and compare with the decoded tag.
5. If the signature is invalid the result is NOT VERIFIED. Otherwise compare each block with the recovered bit and continue with tamper localization.

5.4 Tamper Localization

After a valid signature, each block is marked as disagreeing when its own vote differs from the recovered bit. A 3x3 local average of the disagreement map is thresholded at 0.3, isolated single blocks are removed, and connected regions are labelled. Each region is converted to pixel coordinates and drawn as a bounding box on a heat-map. Unedited areas agree with the payload, whereas an edited area no longer carries the mark and disagrees at roughly random.

5.5 Verdict Rule

Condition	Verdict
No valid signature (unsigned image, wrong key, or mark destroyed)	NOT VERIFIED
Valid signature and flagged area of 2% or less	AUTHENTIC
Valid signature and flagged area above 2%	TAMPERED

Table 3: Verdict decision rule

6. Implementation

6.1 Technology Stack

Component	Technology	Purpose
Language	Python 3.13	Implementation
Numerics	NumPy [13], SciPy [14]	Array maths, DCT, filtering, labelling
Imaging	Pillow	Image I/O, colour conversion, JPEG
Transform	Haar DWT (own code), block DCT	Frequency decomposition
Security	hashlib, hmac (HMAC-SHA256)	Signature and key derivation
Interface	Streamlit, argparse	Web UI and command line
Evaluation	Matplotlib, Pytest	Charts, 11 unit tests

Table 4: Technology stack

6.2 Source Modules

File	Role
authmark/core.py	Embedding and verification engine, payload and HMAC, tamper localization
authmark/attacks.py	JPEG, noise, blur, brightness, rescale, crop and patch-paste attacks
authmark/metrics.py	PSNR and SSIM
authmark/report.py	Summary tables, tamper overlay and report image
cli.py, app.py	Command line and Streamlit web interface
evaluate.py, tests/	Benchmark script and unit tests

Table 5: Source modules

Source code: https://github.com/GAURISHTODI/-AuthMark-AI-Image-Authenticity-Verification.git

7. Results and Discussion

7.1 Test Setup

Eight synthetic 512 x 512 colour images were signed with a fixed key. Every attack was applied to all eight signed images and each result was verified with the same key. Un-watermarked images and wrong-key checks (16 cases) were used to measure false accepts. Local edits were simulated by pasting a noisy patch over 9% and 4% of the image area.

7.2 Imperceptibility

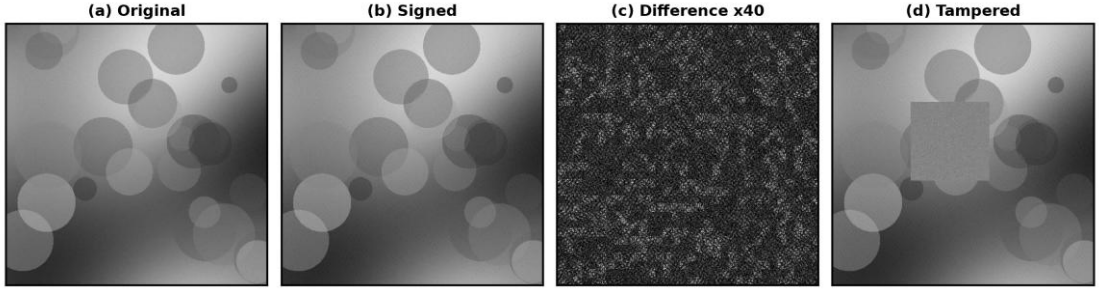


Figure 4: Original, signed, difference map (amplified 40 times) and tampered image

Metric	Value
Mean PSNR (8 images)	42.30 dB
Mean SSIM (8 images)	0.9828

Table 6: Imperceptibility results

The signed image cannot be distinguished from the original by eye; only the amplified difference map shows the mark, which looks like low-amplitude noise spread over the whole image.

7.3 Robustness

Attack	Verified AUTHENTIC	Block agreement
No attack	8 / 8	100.0%
JPEG quality 90	8 / 8	100.0%
JPEG quality 75	8 / 8	100.0%
JPEG quality 50	8 / 8	99.5%
Gaussian noise, sigma 3	8 / 8	100.0%
Gaussian noise, sigma 8	8 / 8	98.7%
Brightness x1.1	8 / 8	100.0%
Rescale 50% and back	8 / 8	100.0%
Blur, radius 0.6	8 / 8	100.0%
Blur, radius 1.0	0 / 8	signature fails
Crop to 90%	0 / 8	66.3%

Table 7: Robustness results (8 images per row)

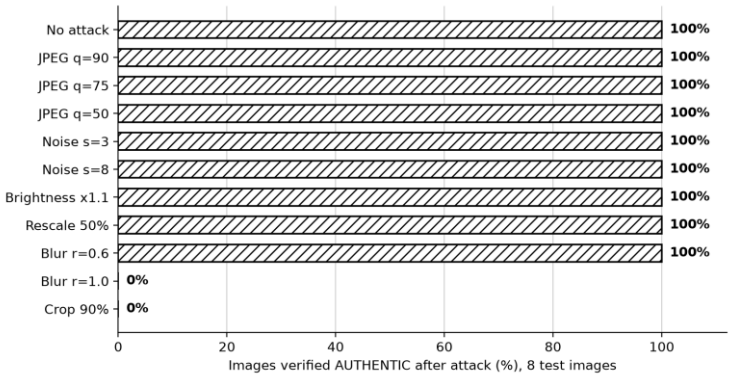


Figure 5: Robustness of the watermark under common attacks

The watermark survives the common operations a shared image undergoes. It fails for cropping, which shifts the block grid, and for blur of radius 1.0, which removes the mid-frequency coefficients carrying the mark. These are stated limits of the present design.

7.4 Tamper Detection

Edit	Detected as TAMPERED	Mean flagged area
Pasted patch, 9% of image	8 / 8	6.2%
Pasted patch, 4% of image	8 / 8	2.8%
Un-watermarked or wrong key (false accepts)	0 / 16 accepted	not applicable

Table 8: Tamper-detection results

All edits were detected and no unsigned or wrong-key image was accepted. The flagged area is smaller than the true patch area because the edge blocks of a patch can still agree with the payload by chance; the bounding box nevertheless covers the edited region (Figure 9).

7.5 Web Application Demonstration

The Streamlit application has a Sign tab and a Verify tab. A secret key is entered in the sidebar. In the Sign tab an image is uploaded and the signed PNG is produced together with its PSNR and SSIM (Figures 6 and 7).

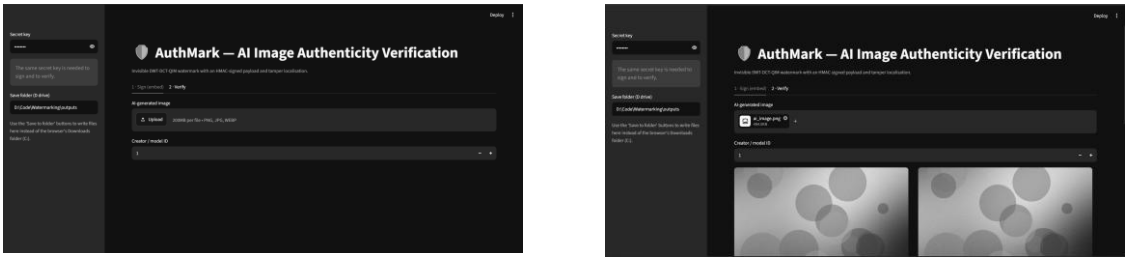


Figure 6: Web application, Sign tab: empty upload form (left) and image uploaded (right)

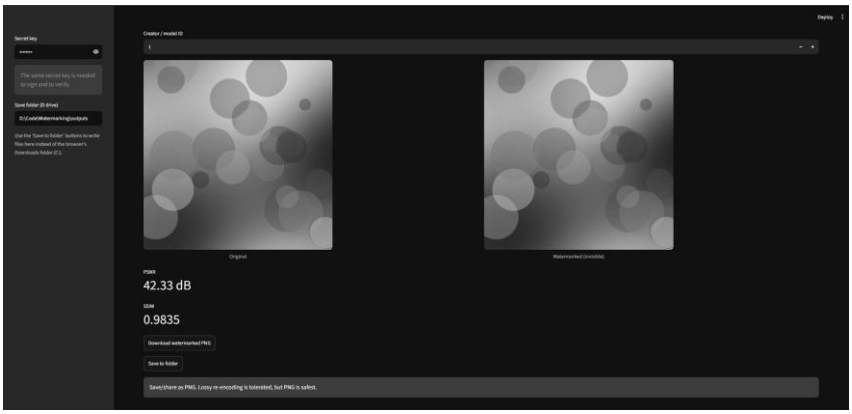


Figure 7: Sign tab results: original and invisible watermarked image with PSNR 42.33 dB and SSIM 0.9835

In the Verify tab a signed image is checked with the same key. Figure 8 shows an AUTHENTIC verdict with the input and tamper map, Figure 9 shows the TAMPERED verdict for the edited image with the edited region boxed, and Figure 10 shows the verification report that lists the payload, extraction quality, tamper analysis and image details.

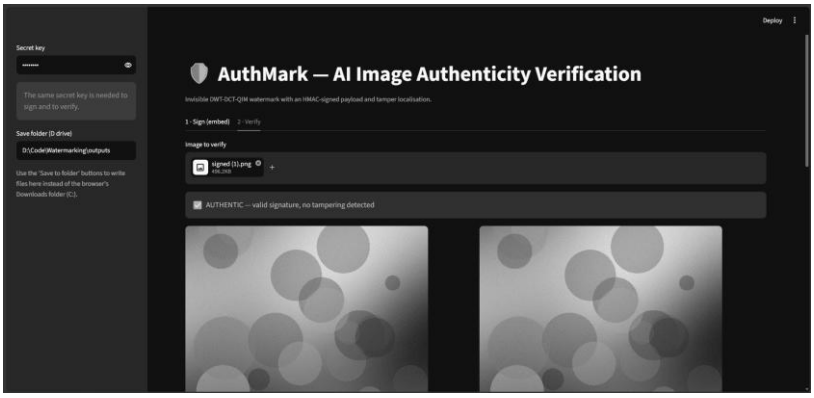


Figure 8: Verify tab: AUTHENTIC verdict for a signed image

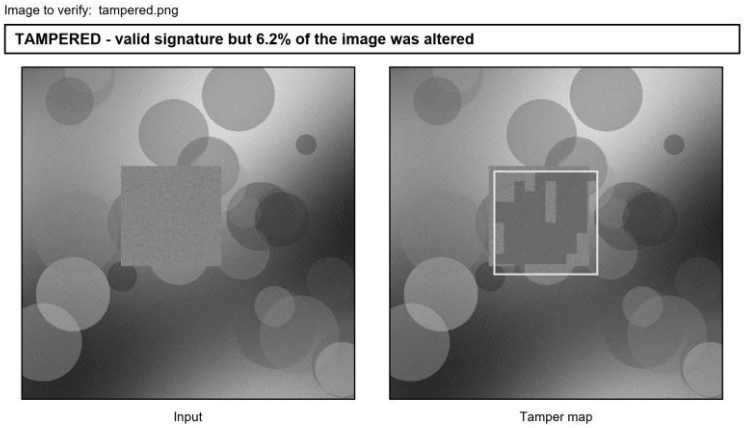


Figure 9: Verify tab: TAMPERED verdict for the edited image, with the input and the tamper map (red overlay, yellow box on the edited region)

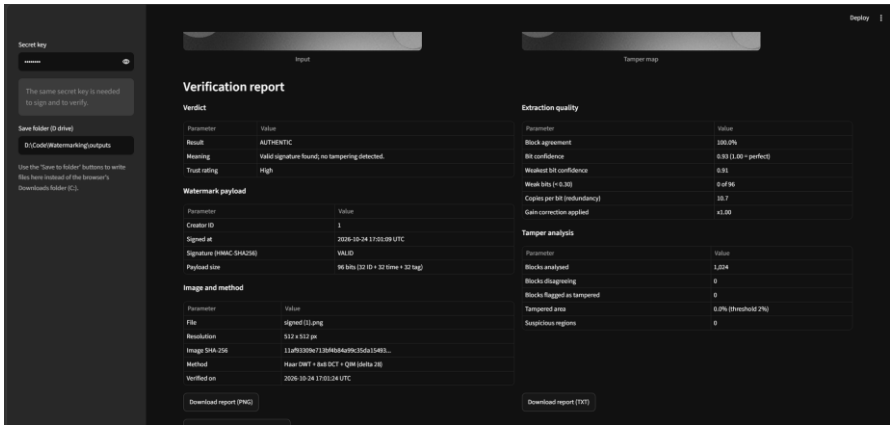


Figure 10: Verification report tables (verdict, payload, extraction quality, tamper analysis, image and method)

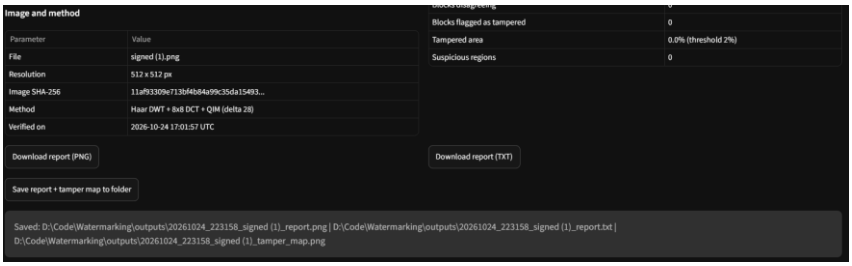
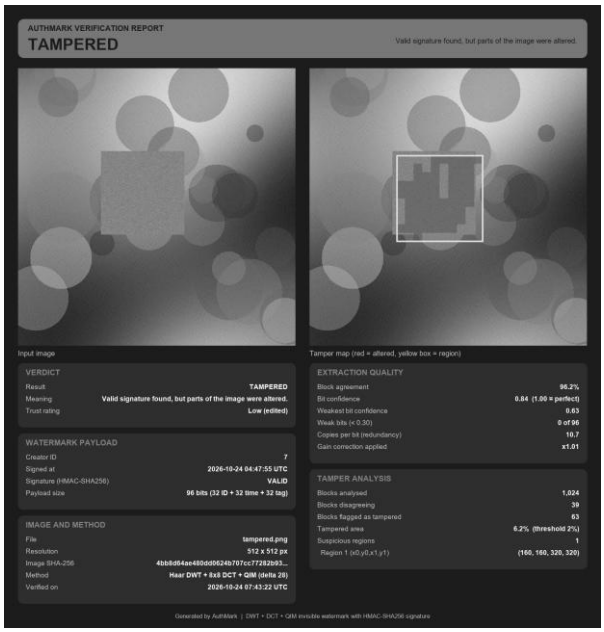


Figure 11: Report, text report and tamper map saved to a chosen folder on another drive

For a tampered image the report shows a TAMPERED verdict, a valid signature, 96.2% block agreement, 6.2% of the area flagged and the coordinates of the edited region. The report can be exported as an image or as plain text (Figure 12).



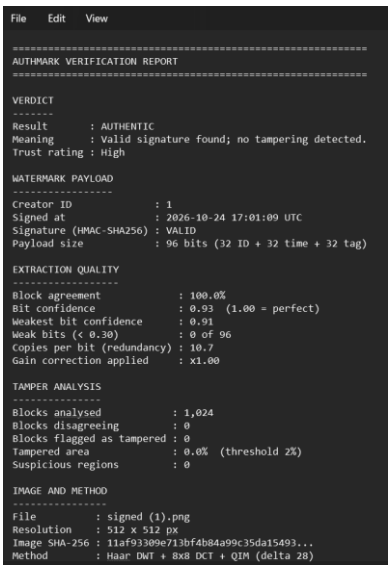


Figure 12: Generated report for the tampered image (left) and the text version of a report (right)

7.6 Discussion and Limitations

- Geometric changes such as cropping and rotation break block alignment; a synchronization pattern would be needed.
- Strong blur removes the mid-frequency mark, and regeneration of the whole image by a diffusion model removes any pixel-domain mark.
- Verification needs the secret key; a public-key signature would allow open verification but requires a longer payload.
- The evaluation uses synthetic images; testing on real photographs and diffusion outputs is future work.
- Signed images should be stored as PNG. JPEG recompression is tolerated, but the embedding itself is done on 8-bit pixels.

8. Conclusion and Future Work

AuthMark demonstrates that a single invisible watermark can prove who created an AI-generated image, survive normal sharing, and show where the image was edited. The system embeds an HMAC-signed 96-bit payload with DWT, DCT and QIM, reaches 42.30 dB PSNR, verifies after JPEG compression down to quality 50, noise, rescaling and light blur, detects all localized edits in the tests, and produced no false accepts. It is delivered as a command line tool, a web application and a printable report.

Future work: (i) replace the HMAC with an Ed25519 signature for public verification; (ii) add a synchronization template to resist cropping and rotation; (iii) compare against learned encoder-decoder watermarking [4]; (iv) evaluate on real photographs and diffusion-model outputs; (v) offer a batch or web-service interface.

References

[1] I. J. Cox, J. Kilian, F. T. Leighton and T. Shamoon, "Secure spread spectrum watermarking for multimedia," IEEE Transactions on Image Processing, vol. 6, no. 12, pp. 1673-1687, 1997.

[2] B. Chen and G. W. Wornell, "Quantization index modulation: A class of provably good methods for digital watermarking and information embedding," IEEE Transactions on Information Theory, vol. 47, no. 4, pp. 1423-1443, 2001.

[3] M. Barni, F. Bartolini, V. Cappellini and A. Piva, "A DCT-domain system for robust image watermarking," Signal Processing, vol. 66, no. 3, pp. 357-372, 1998.

[4] J. Zhu, R. Kaufman, M. Agrawala and L. Fei-Fei, "HiDDeN: Hiding data with deep networks," in Proc. European Conference on Computer Vision (ECCV), 2018.

[5] P. Fernandez, G. Couairon, H. Jegou, M. Douze and T. Furon, "The Stable Signature: Rooting watermarks in latent diffusion models," in Proc. IEEE/CVF International Conference on Computer Vision (ICCV), 2023.

[6] Y. Wen, J. Kirchenbauer, J. Geiping and T. Goldstein, "Tree-Ring watermarks: Fingerprints for diffusion images that are invisible and robust," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[7] H. Krawczyk, M. Bellare and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, IETF, 1997.

[8] Coalition for Content Provenance and Authenticity (C2PA), "C2PA Technical Specification," https://c2pa.org. [Placeholder: add version number and access date]

[9] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600-612, 2004.

[10] I. Daubechies, Ten Lectures on Wavelets, SIAM, Philadelphia, 1992.

[11] N. Ahmed, T. Natarajan and K. R. Rao, "Discrete cosine transform," IEEE Transactions on Computers, vol. C-23, no. 1, pp. 90-93, 1974.

[12] Streamlit Inc., "Streamlit documentation," https://docs.streamlit.io. [Placeholder: add access date]

[13] C. R. Harris et al., "Array programming with NumPy," Nature, vol. 585, pp. 357-362, 2020.

[14] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," Nature Methods, vol. 17, pp. 261-272, 2020.

[15] [Placeholder: course textbook or lecture notes for Digital Watermarking and Steganography, Dr. Diviya M.]

[16] [Placeholder: add any further papers or standards used by the team]

Appendix A: Reproduction Steps

Repository: https://github.com/GAURISHTODI/-AuthMark-AI-Image-Authenticity-Verification.git [Placeholder: add final repository link if it changes]

1. Install dependencies: pip install -r requirements.txt
2. Run the tests: python -m pytest -q
3. Run the benchmark: python evaluate.py (writes results/report.md and results/robustness.png)
4. Sign an image: python cli.py embed samples/ai_image.png samples/signed.png --key k --creator 7
5. Verify: python cli.py verify samples/signed.png --key k
6. Verify the tampered image with a report: python cli.py verify samples/tampered.png --key k --report results/tamper_report.png
7. Start the web application: python -m streamlit run app.py